

0.22.1. Remonter : `deplace_y` et un pas négatif — de base, ailleurs

Le 0.22 en colonne 10, avec le B : descente et montée.

1. Ce qu'il faut comprendre

C'est le 0.22, avec une autre lettre, le B (`ALPHABET[1]`, la 2e lettre), et une autre place : la **colonne 10**, dans les deux `deplace_y`.

C'est la **version de base** : une seule lettre, à la deuxième place. Le 0.22.2 met les deux ensemble.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <deplace_y>` → [0.20.3. La fonction `deplace\_y\(\)` — avancer sur une colonne](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1094 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      63 o ← la console, à l'allumage - la mise en route de la console
deplace_y()    158 o ← main() - une fonction fournie par le langage
main()         51 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AttendreAcces  12 o ← EffacerCases - attend de pouvoir écrire une case
AdresseCase    10 o ← poser(), effacer() - trouve l'adresse d'une case de l'écran
EffacerCases   12 o ← effacer() - efface des cases
CopierTuiles   18 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Tuiles        704 o ← CopierTuiles - la police entière
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
poser()      x 2   26 o sur place, dans deplace_y() + AdresseCase (10 o)
effacer()    x 1   15 o sur place, dans deplace_y() + AdresseCase (10 o), EffacerCases (12 o)
image()      x 2    6 o sur place, dans deplace_y(), main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
deplace_y() x 2   40 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <deplace_y> // fait avancer une tuile sur sa colonne
#include <ALPHABET>  // les lettres de la police : ALPHABET[0] est le A
```

2. UNE VARIABLE — PAS

```
uint8_t pas = 5;    // +5 : descend ; -5 : monte
```

3. UNE VARIABLE — Y

```
uint8_t y = 0;      // la LIGNE de la lettre : elle part de 0 (tout en haut)
```

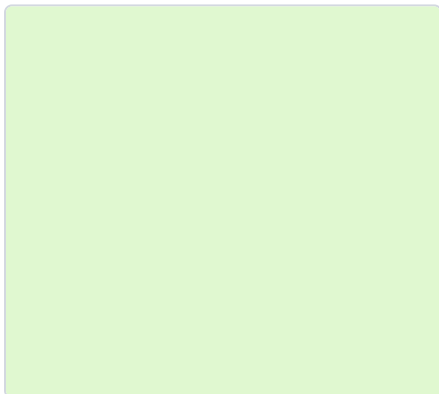
4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {        // Le programme commence ici.
    // deplace_y(colonne, ligne, tuile, pas) : la tuile descend (pas positif) ou
    // monte (pas négatif), puis la fonction REND la ligne d'arrivée.
    // Le changement : la colonne 10 et la lettre B (ALPHABET[1]).
    y = deplace_y(10, y, ALPHABET[1], pas);    // descente : y vaut 5
    y = deplace_y(10, y, ALPHABET[1], -pas);    // montée : y vaut 0

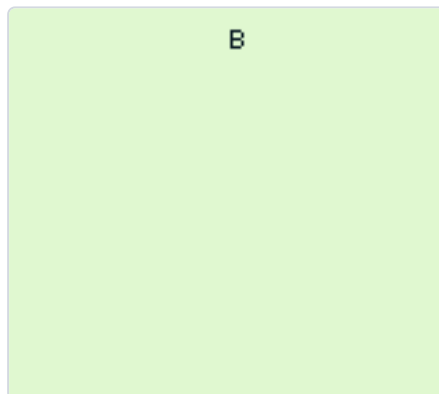
    while (true) {    // La boucle de jeu, pour toujours :
        image();      // attendre l'image suivante. Le B est revenu en haut.
    }
}
```

3. Ce qui se passe, étape par étape

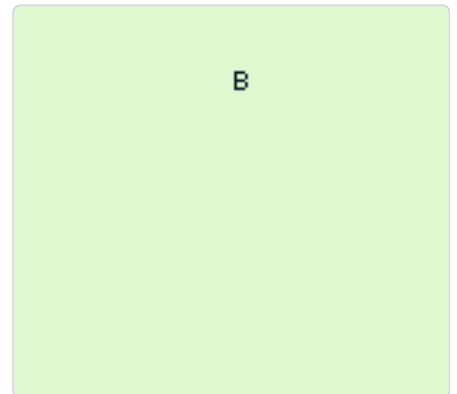
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



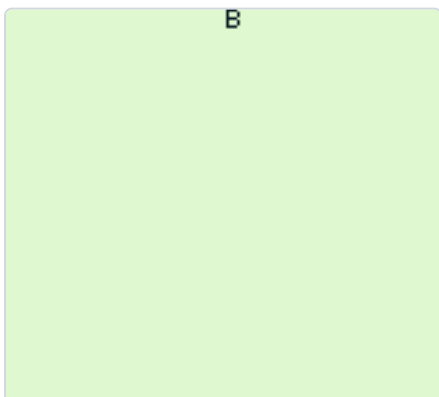
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Un B qui descend puis remonte, en colonne 10.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le B passe par (10, 5)

✓ le B finit en (10, 0) (10,0)

Ensuite : **0.22.2. Remonter : `deplace_y` et un pas négatif — doublé, deux positions** — chapitre 0 : Avant tout. · [gameboy3](#) — écrire une cartouche Game Boy en C++